CS5383 · PRUEBAS Y VERIFICACIÓN DE SOFTWARE · PROYECTO 2 · ENTREGA 3

Rendimiento de Qolqe Wallet

Carga, pico, estrés y resistencia con k6 y Docker frente al requerimiento «Quincena Qolqe»

Johar Jared Barzola Estrella · UTEC · noviembre de 2026

U-01 · p95 < 800 ms

100
simultáneos en hora punta · M-03

300
en la campaña, el triple · M-09

¿N?
capacidad: la mide el estrés · U-08

AGENDA · 15 MIN DE EXPOSICIÓN CON DEMO · 5 DE PREGUNTAS

Cinco partes, en el orden del enunciado

01	**Herramienta y entorno** k6 frente a JMeter, Gatling, Locust, Artillery y Taurus · Docker con 1 CPU y 1 GB	1 min
02	**Flujos críticos, umbrales y supuestos** F-01 a F-06 · U-01 a U-09 · lo que el requerimiento no dice	1.5 min
03	**Modelo de carga y datos** de dónde sale cada valor · escenarios · un usuario por VU	2 min
04	**Demostración en vivo** correr.sh demo: hora punta real y «estrés disfrazado»	3 min
05	**Resultados frente a los umbrales** p95, errores, throughput, capacidad, cuello y recomendaciones	5 min

EL REQUERIMIENTO · ENUNCIADO §4

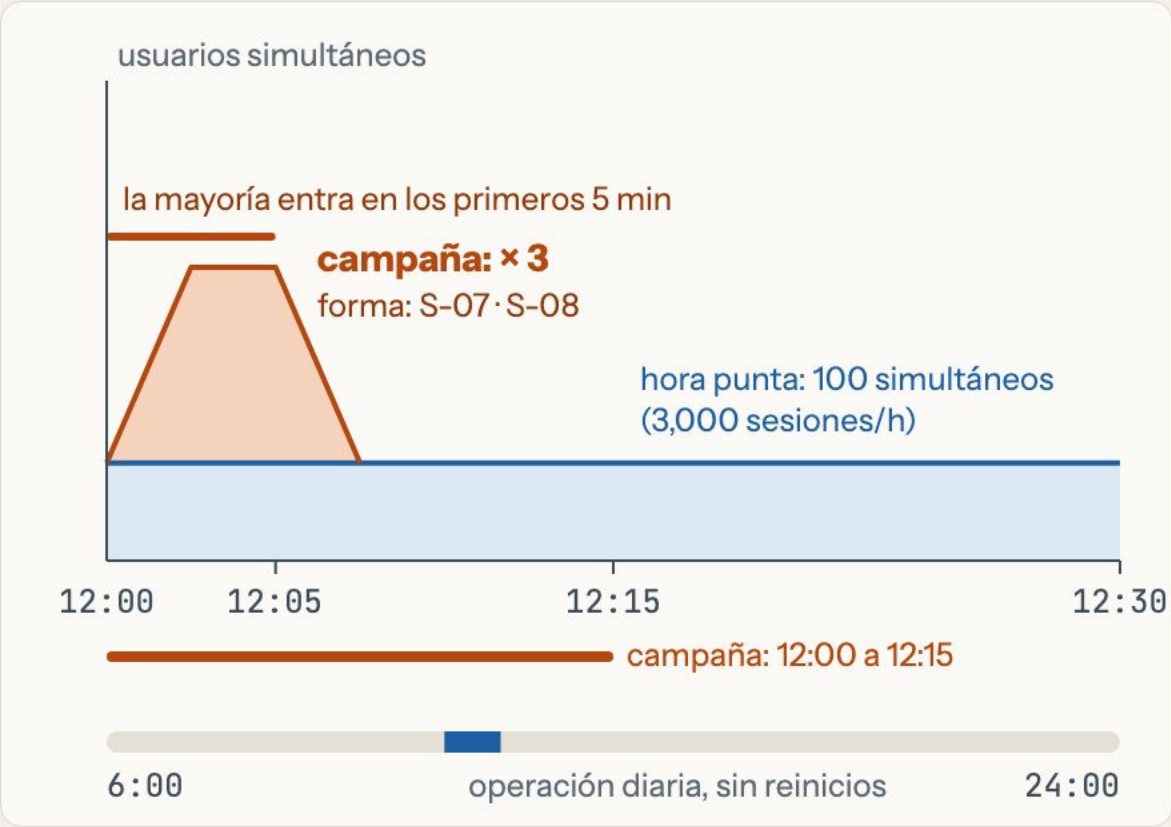
Quincena Qolqe: lo que el negocio necesita saber

1 **¿Cumplimos en hora punta?**
12:00–13:00 · 3,000 sesiones/h · sesiones de 2 min con login y 6 operaciones

2 **¿Y en el pico de la campaña?**
días 15 y último, 12:00–12:15 · el triple de simultáneos · la mayoría entra en 5 min

3 **¿Hasta cuántos usuarios simultáneos podemos crecer?**
antes de dejar de cumplir lo que se espera del servicio

Además: Qolqe opera todos los días de 6:00 a 24:00 sin reinicios.



k6 2.3.0 frente a cinco alternativas

Criterio	k6	JMeter	Gatling	Locust	Artillery	Taurus
C1 Modelo abierto y retraso con umbral	Sí	Parcial	Parcial	No	Parcial	Parcial
C2 Umbral del SLA con código de salida	Sí	No	Sí	Parcial	Sí	Sí
C3 Reporte HTML local y datos crudos	Sí	Sí	Parcial	Parcial	Parcial	Parcial
C4 Liviano en la misma laptop	Sí	No	Parcial	Parcial	Parcial	No
Criterios cumplidos	4 de 4	1	1	0	1	1
Motivo principal	Elegida: cumple los 4	Sale con código 0 aun con fallas	JDK y Maven; no exporta crudos	Sin modelo abierto	Ya no genera HTML local	Capa extra sobre otra herramienta

Calificación (informe §2): Sí si todas las filas del criterio dicen Sí; No si todas dicen No; si no, Parcial

Limitaciones de k6: sin GUI ni grabador (probamos la API) · la mezcla ponderada se programa · no mide el servidor: psutil y docker stats aparte

Ejes de S13: las seis son libres (k6 AGPL-3.0; las demás Apache-2.0, MIT o MPL-2.0) · protocolo, sin el módulo browser · el código 99 corta el job de CI · LoadRunner, NeoLoad, BlazeMeter y Silk Performer: comerciales, descartadas por licencia

Versiones evaluadas (nov-2026): k6 2.3.0 · JMeter 5.6.3 · Gatling 3.16.0 · Locust 2.47.0 · Artillery 2.0.34 · Taurus 1.17.2

01 · HERRAMIENTA — ENTORNO (ENUNCIADO §2 Y §5)

Entorno: Qolqe en Docker con 1 CPU y 1 GB

Local: compite por el CPU | Dos equipos: solo hay una laptop | Codespaces: suma latencia de internet | **Docker: elegido**

Mac M1 Pro · 10 núcleos · 16 GB · macOS

Contenedor qolqe-perf · Docker Desktop

1 CPU 1 GB `--cpus=1 --memory=1g`

uvicorn con 1 proceso · SQLite en volumen · base de 539.5 MB

Medido con psutil sobre el PID 1 (contenedor auxiliar) y docker stats

50 sesiones/min

127.0.0.1:8090

Host macOS

k6 2.3.0

9 núcleos libres · psutil mide el CPU de k6 y del equipo

V-01 · cambió al ejecutar

CPU de k6 < 80 % (decide)

peor ventana: 1.7 %. El CPU del equipo se informa aparte: macOS lo pasó en el pico y el estrés

V-05 · validez

Misma base, proceso nuevo

restaurada y verificada en cada corrida; cpu.max = 100000 100000 (200000 con 2 CPU)

S-17 · enunciado §5

Línea base relativa

no es la capacidad de producción y no se extrapola (S13)

Seis flujos críticos: frecuencia, dinero y riesgo

Frecuencia ≥ 15 % · Mueve dinero · Riesgo alto con evidencia en el código → **crítico si cumple al menos 1**

Flujo	Frecuencia	Dinero	Riesgo y evidencia en el código
F-01 Iniciar sesión	1 por sesión	Indirecto	PBKDF2 de 200,000 iteraciones por login (credenciales.py:13,24)
F-02 Transferir	15 %	Directo	Antifraude duerme 40–120 ms; carrera DEF-21
F-03 Saldo y perfil	30 %	Indirecto	La operación más frecuente
F-04 Recargar celular	10 %	Directo	SCAN de recargas sin índice
F-05 Pagar servicios	4 % + 6 % consulta	Directo	Deudas sin dueño: un pago duplicado da 409
F-06 Movimientos	15 %	No	Consulta N+1: ≈ 243 consultas por petición

Todo flujo de dinero es crítico: tiene umbral propio (1.5 s). Contactos y Otras (10 % cada uno) no cumplen ninguno: van en la mezcla como carga de fondo.

Nueve umbrales: métrica, percentil, límite y carga

MÉTRICA + PERCENTIL + VALOR LÍMITE + CARGA (S14) · U-01 a U-05 con 100 simultáneos (S-01)

ID	Qué mide	Umbral
U-01	p95 de consultas e inicio de sesión	< 800 ms
U-02	p95 de operaciones de dinero	< 1,500 ms
U-03	operaciones fallidas (sin los 422 de negocio)	≤ 1 %
U-04	throughput del estado estable	≥ 9.6 req/s y ≥ 49 sesiones/min
U-05	sin acumular retrasos	0 descartadas · 90–110 simultáneos
U-06	campaña, pico de 300: p95 y error	< 1,600 / 3,000 ms (2×) · ≤ 1 %
U-07	descuadres de dinero, en todas las corridas	= 0
U-08	capacidad, rampa de 100 a 2,400	se mide: cumple U-01 a U-05
U-09	memoria proyectada a un día	RSS a 54,000 sesiones < 700 MB

U-07 es 0, no < 1 %: una operación a medias o duplicada no es lentitud, es un defecto de integridad.

Partes no convertibles y supuesto tomado

S-01

Falta: con qué carga se cumplen los tiempos

En hora punta: 100 simultáneos y 3,000 sesiones/h

S-04

Falta: «sin acumular retrasos» no es un número

Modelo abierto: 0 sesiones descartadas y throughput ≥ 98 % del ofrecido

S-05

Falta: «degradación razonable» no es un número

p95 hasta 2× el normal (1,600 / 3,000 ms), error ≤ 1 % y recuperación

S-06

Falta: qué cuenta como operación fallida

5xx, timeout, conexión o cuerpo incorrecto; los 422 de negocio van aparte

S-07 · S-08

Falta: ¿el triple es máximo o promedio? ¿cuánto es «la mayoría»?

Máximo de 300: 150 sesiones/min de 12:00 a 12:05, el 58 % de la campaña

S-12

Falta: no hay 18 horas para probar antes de la entrega

Resistencia de 60 min y proyección a 54,000 sesiones (18 × 3,000)

De sesiones por hora a usuarios simultáneos

M-01

«se inician unas 3,000 sesiones por hora»

$$\lambda = 3{,}000 \div 3{,}600\ s = 0.833/s$$

M-02

«una sesión dura en promedio 2 minutos»

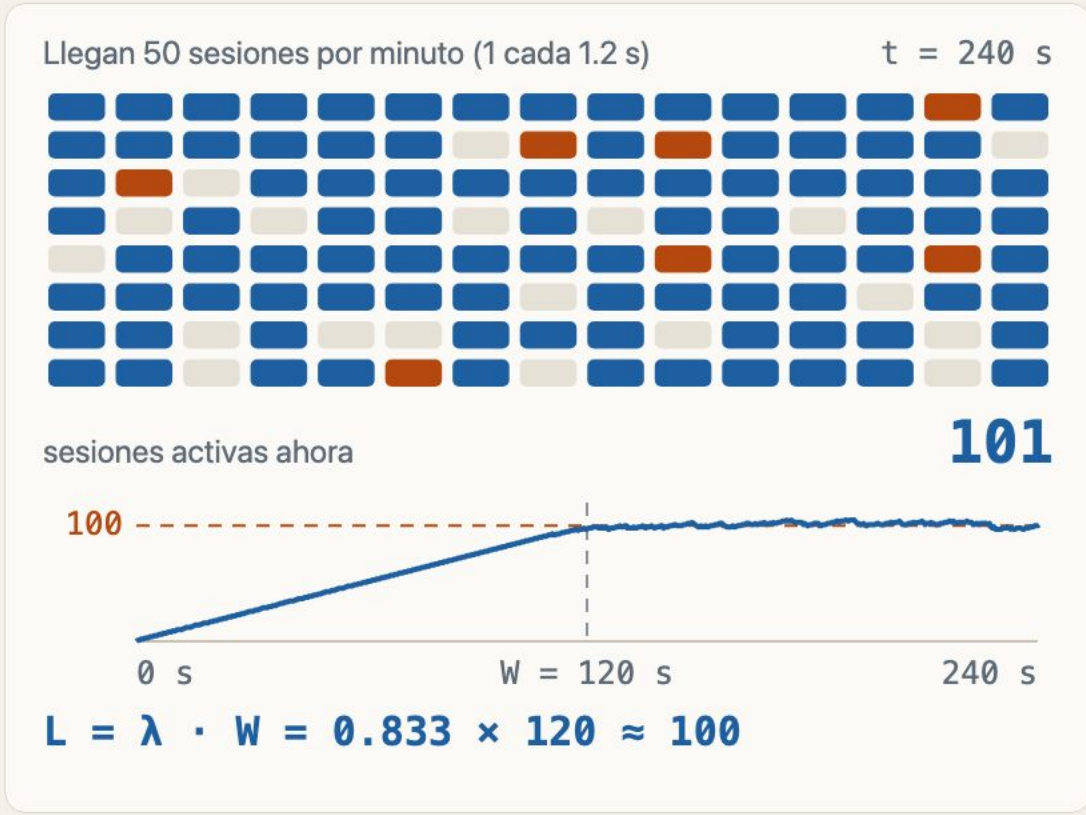
$$W = 120\ s$$

M-03 · Ley de Little (aporte propio)

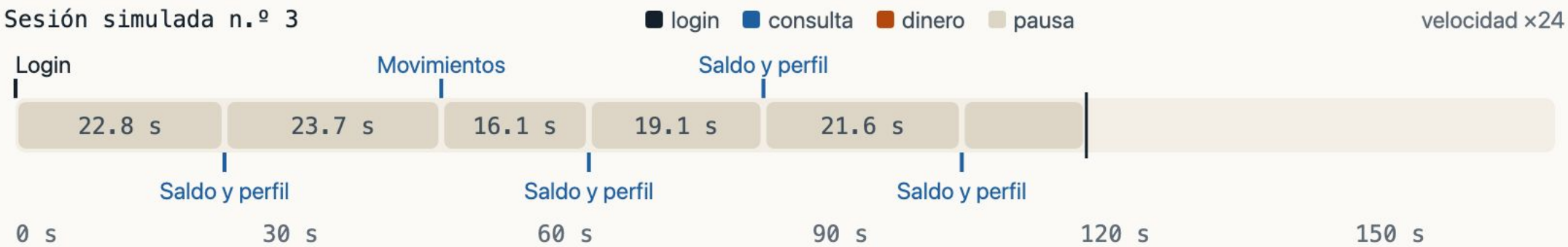
$$L = \lambda \cdot W = 0.833 \times 120$$

= 100 simultáneos

Ramp-up = 120 s (M-24, ID del diseño): la concurrencia se llena sola en lo que dura una sesión. **Campaña:** 3 × 100 = 300 (M-09).



Anatomía de una sesión de 2 minutos



M-04

6 × 18 s = 108 s

6 pausas: tras el login y entre operaciones («de 10 a 25 s, 18 s en promedio»)

M-05 · S-13

120 - 108 = 12 s

cierre sin peticiones: así «2 minutos» y «100 simultáneos» valen lo mismo que en el requerimiento

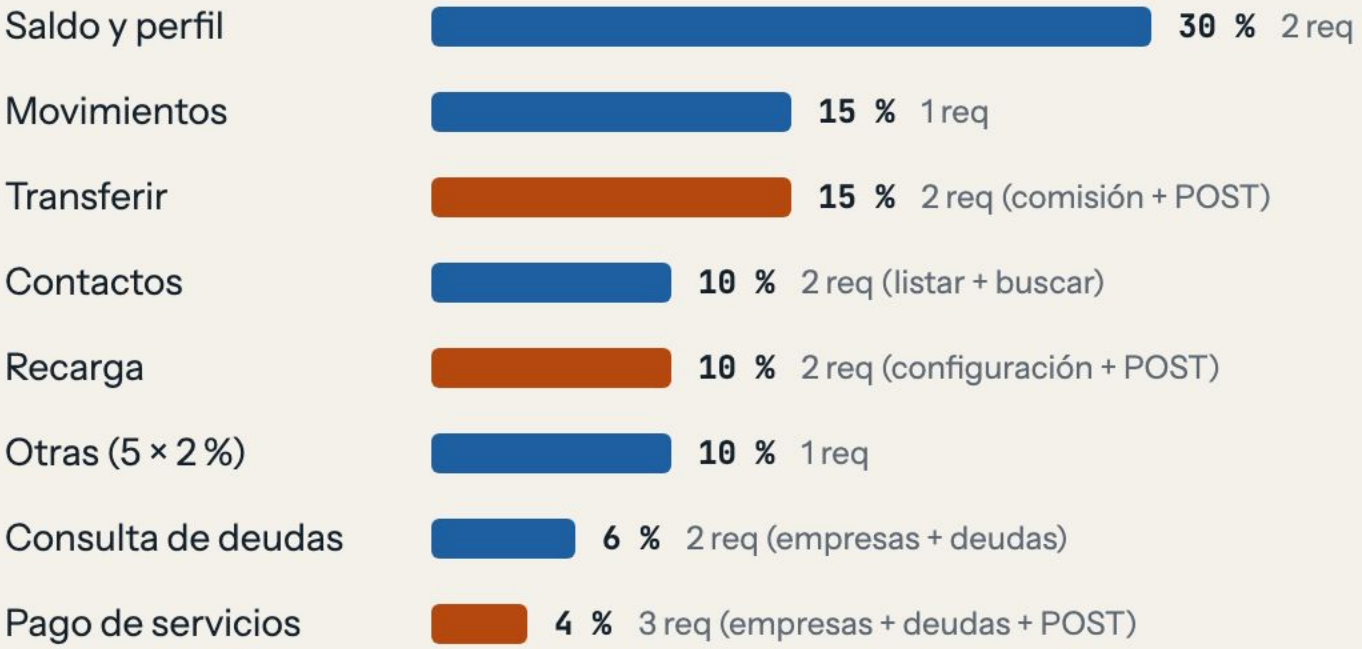
S-14

media = 18 s

triangular(10, 19, 25): rango de 10 a 25 s y media (10 + 19 + 25) ÷ 3; una uniforme daría 17.5 s

Distribución de operaciones y peticiones esperadas

consulta dinero · % de las 6 operaciones · peticiones por operación según app.js



Login: 1 por sesión, fuera del sorteo · 6 operaciones sorteadas con reemplazo (M-06)

peticiones por operación

1.79

promedio ponderado (M-06 × M-07)

peticiones por sesión

11.74

1 + 6 × 1.79

en hora punta (M-08)

9.78 req/s

0.833 × 11.74 · 35,220 por hora

operaciones de dinero

1.45 POST/s

29 % de las operaciones

Cuatro preguntas, cuatro tipos de prueba

Carga · ¿cumplimos en hora punta? S13 · caso A

50 sesiones/min · llenado 120 s · 15 min medidos



Pico · ¿y en la campaña? S13 · caso B

salto ×3 en 30 s · 150 sesiones/min hasta las 12:05



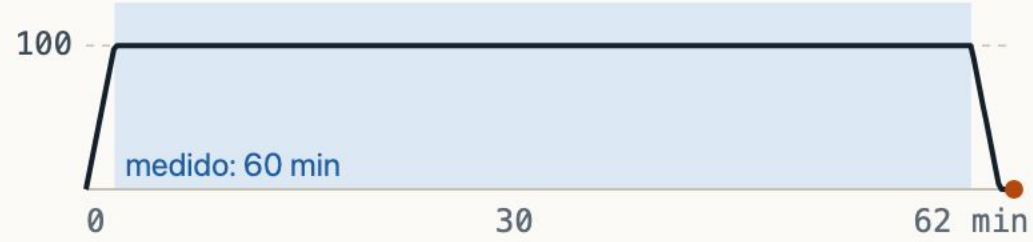
Estrés · ¿hasta cuántos podemos crecer? S13 · caso D

rampa de 100 a 2,400 simultáneos · +100 por minuto



Resistencia · ¿18 h sin reinicios? S13 · caso C

50 sesiones/min por 60 min · 3,000 sesiones → 54,000



Datos de prueba: un usuario distinto por VU

3,000
usuarios distintos: el VU n usa la fila n

S/ 10,000
de saldo inicial por cada usuario

30
deudas propias: el pago nunca choca

1,000
movimientos por usuario (243 en 30 días)

preparar.sh 3000 (datos en 154 s): 15,000 contactos, 90,000 deudas, 3,000,031 movimientos, base de 539.5 MB

Reglas de negocio que no deben fallar por datos

RN-04 · 3 PIN fallidos bloquean → un usuario fijo por VU

RN-06 · límite de S/ 2,000 al día → montos de S/ 1 a 20

R-G1-03 · 5 recargas por número → número aleatorio

V-02 · un 422 de negocio (p. ej. LIMITE_DIARIO_EXCEDIDO) es error de datos

Soporte · POST /api/pruebas/reiniciar y PUT /api/pruebas/saldo

Demo en vivo: hora punta y «estrés disfrazado»

```
$ ESPERA_MAX=20 tests/performance/correr.sh demo · ≈ 2 min 30 s
```

0:00–1:40 · demo_hora_punta

50 sesiones/min con las pausas reales (M-01)

1:40–2:25 · demo_sin_pausa

50 VUs sin think time

Qué se verá · en el ensayo pasó así

Los simultáneos suben solos (Little) y el p95 queda bajo 800 y 1,500 ms

✓ consultas 103 ms · login 106 · dinero 138 · 0 % error

Qué se verá · en el ensayo

Colapso: errores en rojo, no solo lentitud

✗ login 17.9 s · 35 % error

Ensayo 2:25 → k6: 5 de 9 umbrales · ✓ hora punta · ✗ sin pausa → código 99 (el esperado)

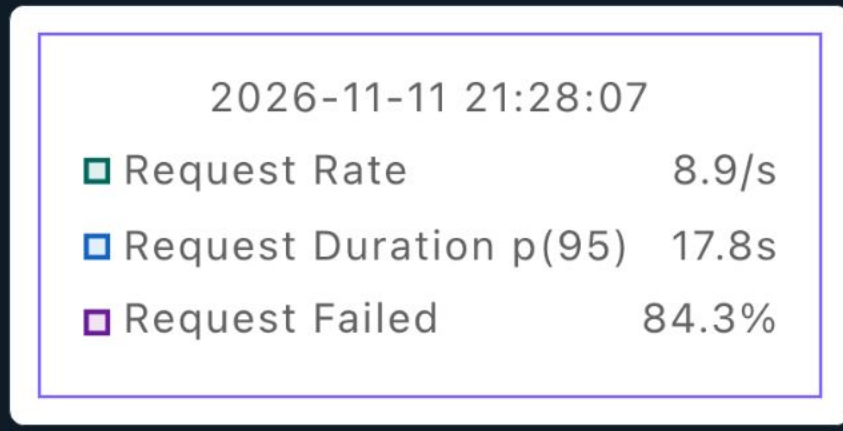
Ensayo 2:30 → verificar.py: 0 descuadres; 17 operaciones cortadas en vuelo, 16 aplicadas y conciliadas

«Sin think time es una prueba de estrés disfrazada» (S13) · ensayo: demo/20261111-2125

Así lo reportó k6 en el ensayo

consola.txt · demo/20261111-2125

```
$ tests/performance/correr.sh demo
Qolqe · escenario demo · 2026-11-11 21:26:07 → 21:28:32
Umbrales (k6 termina con código 99 si alguno falla):
✓ U-01 demo_hora_punta: consultas              p(95)<800      103.0 ms
✓ U-01 demo_hora_punta: login                  p(95)<800      105.7 ms
✓ U-02 demo_hora_punta: dinero                 p(95)<1500     136.7 ms
✓ U-03 demo_hora_punta: operaciones fallidas   rate<=0.01       0.00 %
✗ U-01 demo_sin_pausa: consultas               p(95)<800    15647.8 ms
✗ U-01 demo_sin_pausa: login                   p(95)<800    17857.7 ms
✗ U-02 demo_sin_pausa: dinero                  p(95)<1500   11522.3 ms
✗ U-03 demo_sin_pausa: operaciones fallidas    rate<=0.01      35.07 %
✓ U-05a demo_hora_punta: sesiones descartadas  count==0             0
Veredicto: 5 de 9 umbrales cumplidos con muestras → FALLA
errores de rendimiento 182 · errores de datos 0
level=error msg="thresholds on metrics … have been crossed"
$ echo $?   # codigo_salida_k6.txt
99
```



reporte.html: el mismo dashboard de localhost:5665. A las 21:28:07, p95 de 17.8 s y 84.3 % de peticiones fallidas.

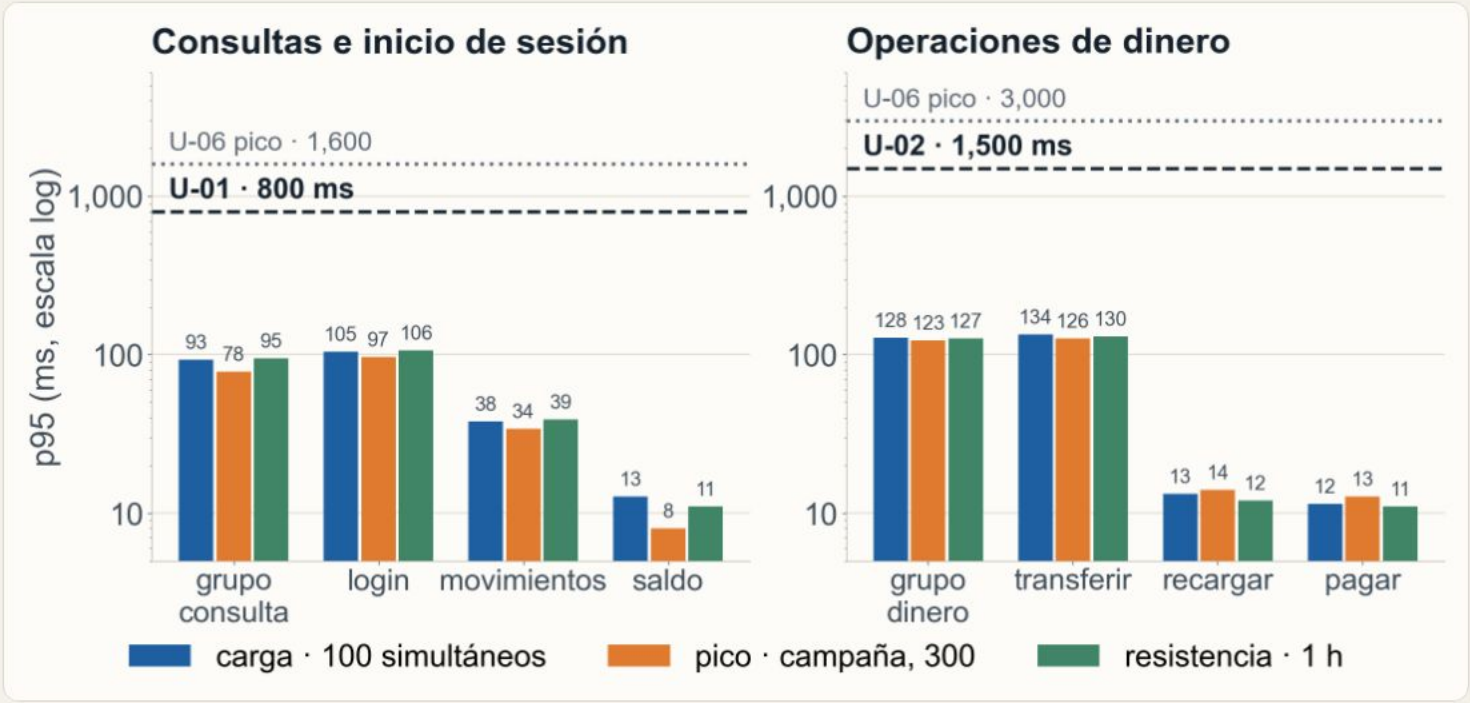
consola.txt y reporte.html · resultados/demo/20261111-2125

Veredicto por umbral: todo lo que se exige, pasa

ID	Criterio	Escenario	Umbral	Medido	Veredicto
U-01	p95 consultas e inicio de sesión	carga · resistencia	< 800 ms	93 · 95 ms	✓ pasa
U-02	p95 operaciones de dinero	carga · resistencia	< 1,500 ms	128 · 127 ms	✓ pasa
U-03	Operaciones fallidas	carga · resistencia	≤ 1 %	0.00 · 0.00 %	✓ pasa
U-04	Sostener el volumen	carga · resistencia	≥ 9.6 req/s	9.77 · 9.77 req/s	✓ pasa
U-05	Sin acumular retrasos	carga · resistencia	0 descartadas	0 · 0	✓ pasa
U-06	Campaña: p95 y error	pico · 304 simultáneos	< 1,600 / 3,000 ms	78 / 123 ms · 0 %	✓ pasa
U-07	Descuadres de dinero	las 8 corridas	= 0	0	✓ pasa
U-08	Capacidad	estrés 1 CPU · 2 CPU	se mide	850 · 1,300	medido
U-09	Día de 18 h sin reinicio	resistencia · 1 h	RSS a 54,000 < 700 MB	92 MB	✓ pasa

Validez: V-06 login 92 frente a 80 ms · V-03 en la demo · V-01 y V-04 cambiados al ejecutar (informe §5)

p95 frente al umbral: muy por debajo



login · el más caro · U-01

105 ms

13 % de 800 ms · pico 97 ms

transferir · dinero · U-02

134 ms

9 % de 1,500 ms · pico 126 ms

campaña · 304 simultáneos · U-06

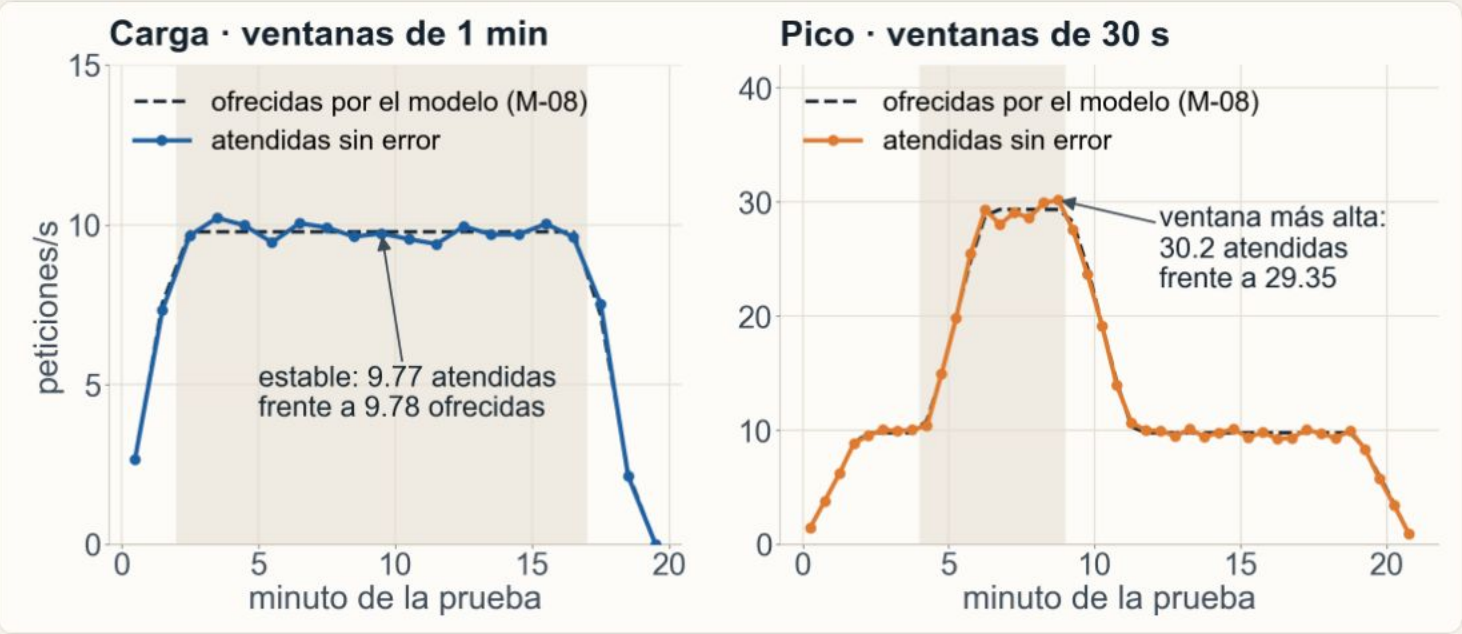
78 / 123

ms · recuperación 80 / 123 ms

Con 100 simultáneos (3,000 sesiones/h), el p95 de transferir fue **134 ms** frente a 1,500 ms, con **0.00 %** de error.

veredicto.md · carga/20261111-1833 · pico/20261111-1918 · resistencia/20261111-1958

Throughput: atiende todo lo que se le ofrece



carga · U-04 (≥ 9.6)

9.77 req/s

ofrecidas: 9.78 req/s (M-08)

sesiones/min · U-04 (≥ 49)

49.6

llegada programada: 50/min (M-01)

pico · ventana más alta

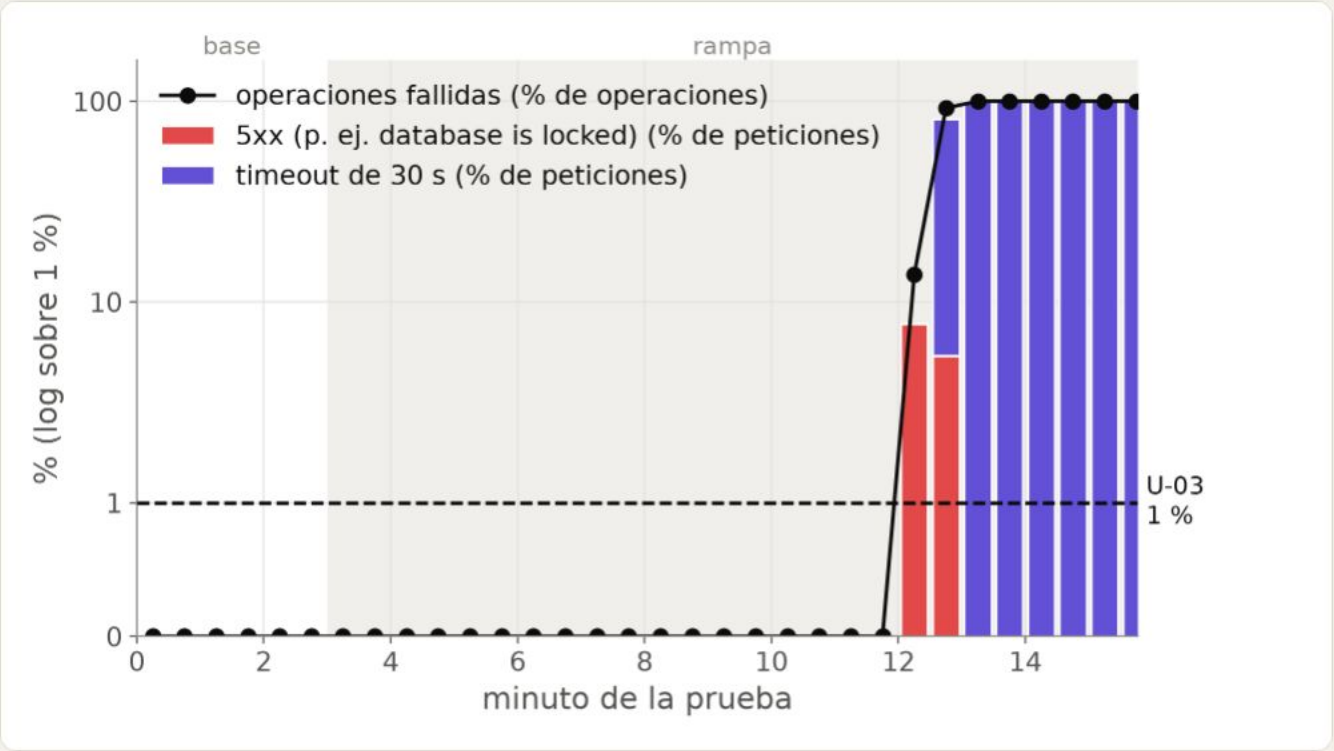
30.2 req/s

ofrecidas en el salto: 29.35

U-05, sin acumular retrasos: 0 sesiones descartadas · 0 de 15 ventanas de 1 min sobre el umbral · simultáneos de 98.8 a 101.0 (Little espera 90 a 110)

ventanas.csv y veredicto.md · carga/20261111-1833 · pico/20261111-1918

Errores: rendimiento y negocio, por separado



hora punta · U-03 (≤ 1 %)

0.00 %

carga, pico, resistencia y demo

negocio 4xx (422…) · V-02 (= 0)

0 en todas

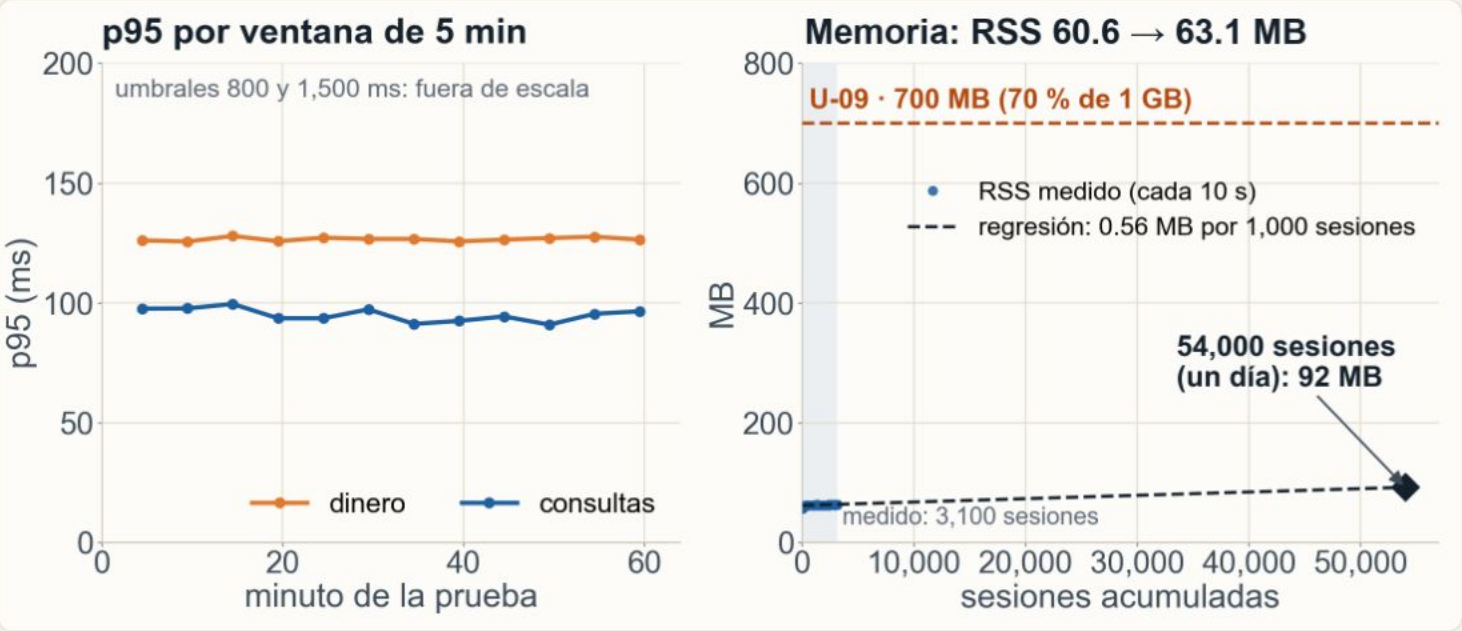
ningún 422 (p. ej. LIMITE_DIARIO_EXCEDIDO)

Al pasar el límite	Respuestas 500	Timeouts o conexión
Estrés·1 CPU	116	4,762
Estrés·2 CPU	120	9,960
Demo sin pausa	101	81 (conexión)

Estrés 1 CPU: los errores aparecen recién al pasar la capacidad (946 simultáneos)

veredicto.md de cada corrida · panel superior de estres/20261111-1940/graficos/G3_errores.png

Resistencia: una hora estable, sin fuga de memoria



U-09 · RSS a 54,000 sesiones

92 MB

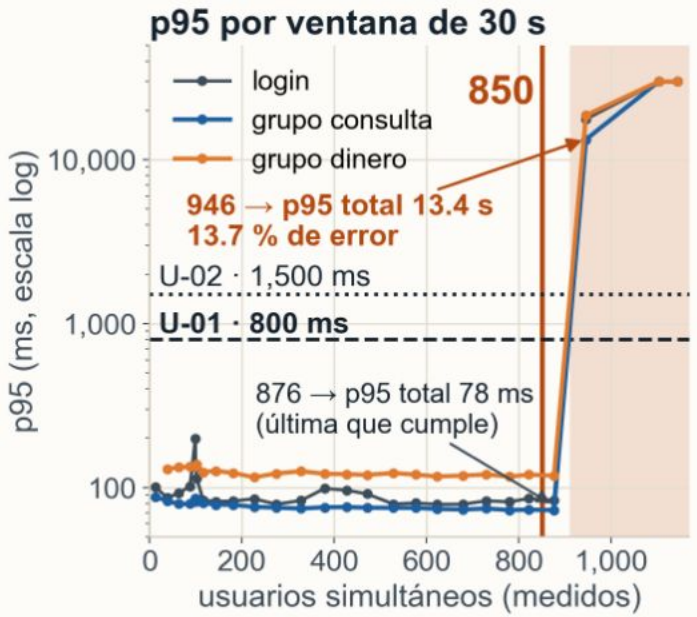
límite 700 MB (70 % de 1 GB) · 0.56 MB por 1,000 sesiones

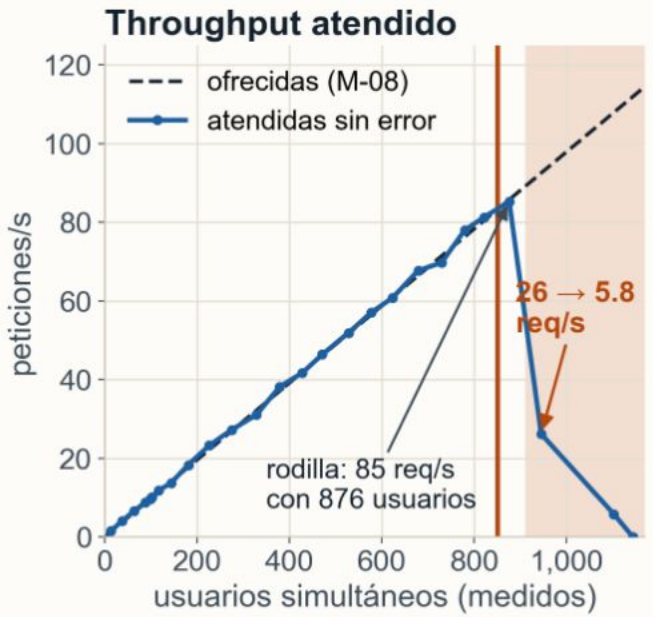
12 de 12 ventanas de 5 min cumplen U-01 a U-03
Deriva del p95: 0.98× consultas · 1.01× dinero
Hilos 7.7 → 7.3 · **descriptores** 42.8 → 42.7

RSS 60.6 → 63.1 MB por ventana de 5 min en 3,101 sesiones (la regresión de U-09 usa n = 3,100, veredicto.md): proyectado a un día, 92 MB. k6 no pasó de 0.2 % del CPU del equipo (V-01). Recursos de todas las corridas: Anexo C.

ventanas.csv · veredicto.md · resistencia/20261111-1958

Estrés con 1 CPU: plano... y luego un muro





Capacidad (U-08) · 1 CPU

850

simultáneos (U-01 a U-05, a 50, S-10): ≈ 8.5× la hora punta y 2.8× el pico de 300

Colapso brusco, no gradual

876 → p95 78 ms (cumple)

946 → p95 13.4 s · 13.7 %

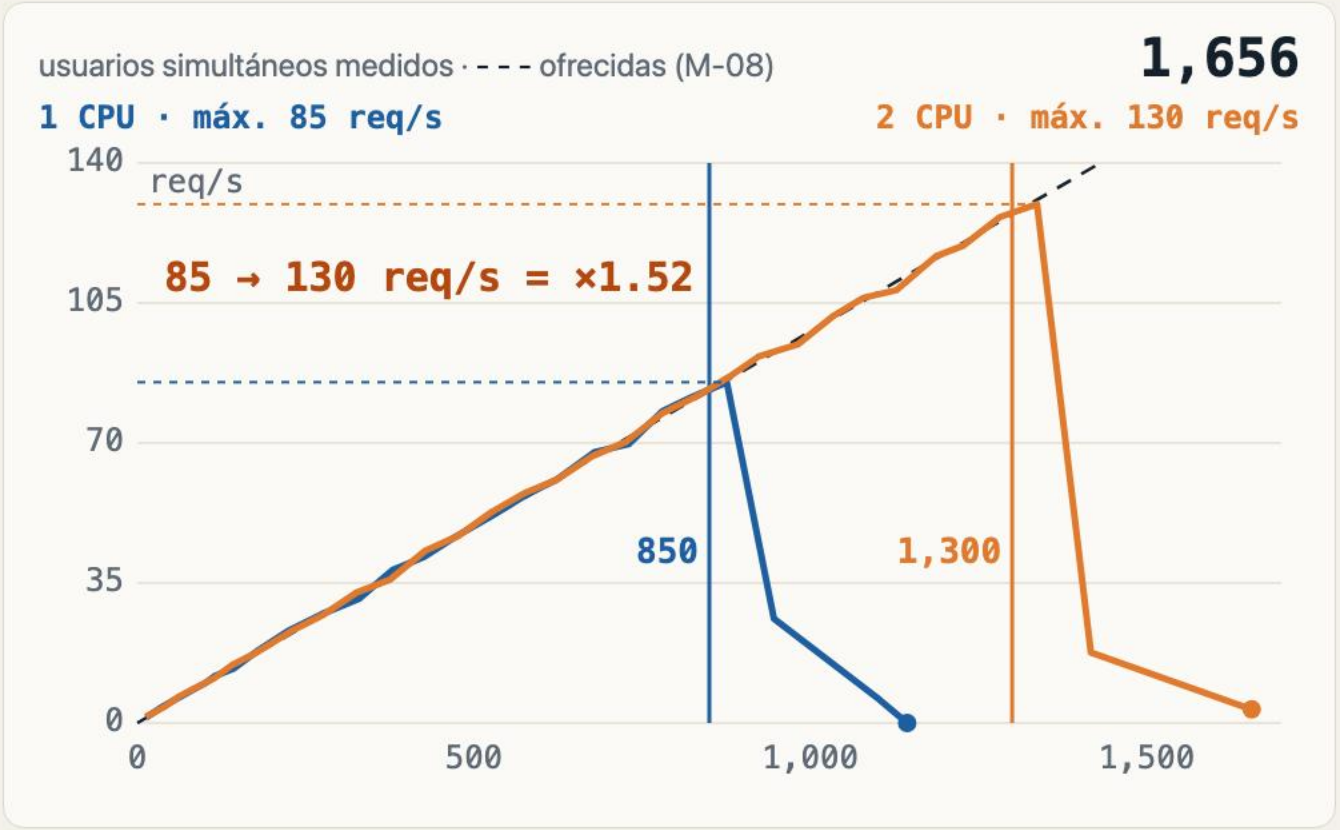
1,103 → 92.5 % de error

Throughput máximo: **85 req/s** con 876 usuarios; después cae a 26 y a 5.8

k6 cortó la corrida a los 947.7 s con su red de seguridad (20 % de fallas acumulado; máximo medido: 1,144 simultáneos): cruzamos el quiebre, sin meseta. Hipótesis del diseño, 1,500 ± 300: **refutada**. Línea base relativa.

tabla_estres.md · ventanas.csv · estres/20261111-1940

¿Doble CPU, doble capacidad? ×1.53, no ×2



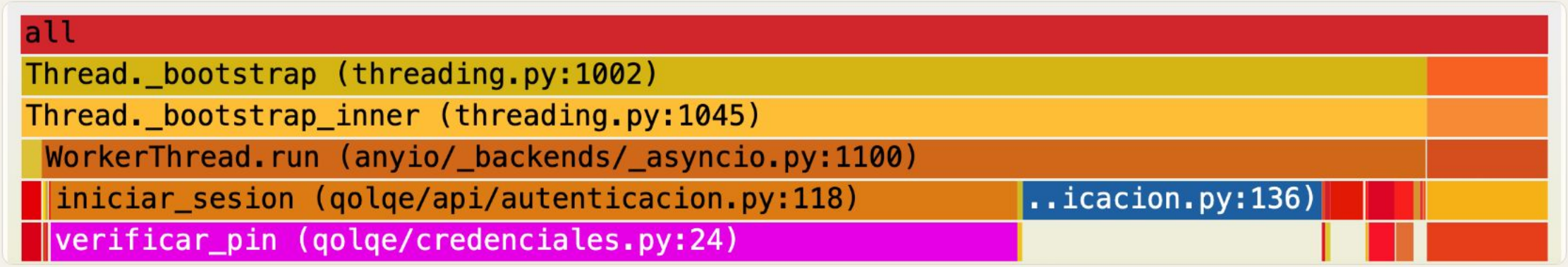
	1 CPU	2 CPU
Capacidad U-08	850	1,300
Throughput máx.	85 req/s	130 req/s
CPU máx. antes del colapso	75.5 % de 100	113 % de 200
Throttling (períodos)	525	217
RSS máx.	478 MB	719 MB

Si el cuello fuera solo el CPU, doblarlo daría ≈ ×2 y caería con el CPU lleno.

Con 2 CPU cae con el 44 % del CPU libre (con 1 CPU, con el 24.5 %): hay otro límite.

tabla_estres.md y ventanas.csv · estres/20261111-1940 · escalabilidad/20261111-2103

Cuello 1: el CPU se va en el PBKDF2 del login



py-spy · estrés con 1 CPU, 790–850 s (ya en el colapso) · 2,518 muestras · ancho de cada barra = % del tiempo

63 %
de las muestras en **verificar_pin**: el hash PBKDF2 del login

53 %
ya en la base (60–120 s, ~60–90 usuarios); 59 % con 2 CPU

19.6 %
esperando el bloqueo de SQLite en el INSERT de sesión

525
períodos de throttling (63.5 s): la cuota de 1 CPU se agota en ráfagas

H1 confirmada: el PBKDF2 domina el CPU. **H2 secundaria:** la N+1 de movimientos pesa 5.7 % en la base (línea 118; ≈ 7 % todo el archivo). Pero el CPU no explica el muro: eso es el cuello 2.

Cuello 2: el muro lo pone SQLite, un solo escritor

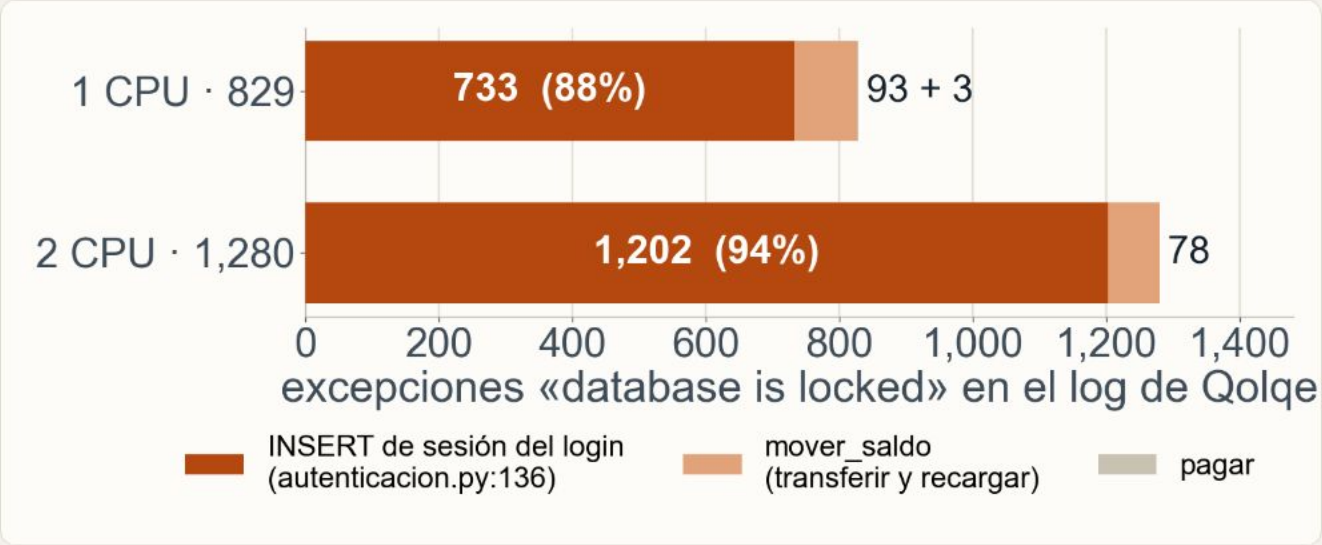
hilos_meseta.txt · escalabilidad/20261111-2103

```
# estrés con 2 CPU · segundo 1,250 (colapso)
$ py-spy dump --pid 1
Process 1: python -m uvicorn qolqe.main:app …

Thread 91 (idle): "AnyIO worker thread"
    iniciar_sesion (qolqe/api/autenticacion.py:136)
    WorkerThread.run (anyio/_backends/_asyncio.py:1100)
Thread 92 (idle): "AnyIO worker thread"
    iniciar_sesion (qolqe/api/autenticacion.py:136)
    WorkerThread.run (anyio/_backends/_asyncio.py:1100)
… 38 hilos más en la misma línea

40 de 45 hilos en autenticacion.py:136
= el pool de AnyIO completo
```

foto de hilos en el colapso, 2 CPU



qolqe.log: 88 % y 94 % de los «database is locked» vienen del login

1 · login: PBKDF2 gasta CPU → **2** · INSERT de la sesión en SQLite → **3** · un solo escritor: espera hasta 10 s → **4** · los 40 hilos del pool, ocupados → **5** · todo en cola: timeout de 30 s y 500

H1 CPU ✓ · H2 N+1 · secundaria · H3 pool ✓ · H4 SQLite ✓ · H5 fuga ✗ · H6 generador ✗

Integridad del dinero, incluso en el colapso

0

descuadres en C1 a C6, en las 8 corridas analizadas

y en el pico descartado. U-07 exige 0: nada a medias ni aplicado dos veces.

Corrida	201 de k6	Filas en BD	Sin confirmar
Carga	1,461	1,461	0
Pico	2,503	2,503	0
Resistencia	5,392	5,392	0
Estrés 1 CPU	3,839	3,841	2 por timeout
Estrés 2 CPU	8,188	8,190	2 por timeout
Demo	102	118	16 aplicadas (17 cortadas)

«Sin confirmar» = se aplicaron, pero el cliente no recibió la respuesta (timeout o sesión cortada por k6). k6 nunca reintenta: no hay duplicados.

`verificacion_dinero.txt de cada corrida · C6: filas en BD - 201 ≤ sin respuesta + cortadas`

Recomendaciones, en orden de prioridad

#	Acción	Ataca
1	**Sacar el INSERT de sesión del bloqueo de escritura:** commit justo después del INSERT, sesión en otro almacén o una BD con varios escritores	el muro (H4)
2	**Fallar rápido en vez de encolar:** busy_timeout bajo y 503 + Retry-After	la cola de 30 s (H3)
3	**Menos PBKDF2 por usuario:** renovar el token en vez de repetir el login, y escalar CPU	el CPU (H1)
4	**Corregir la N+1 de movimientos:** SUM() OVER y paginación en SQL	CPU sobrante (H2)
5	**Clave de idempotencia** en transferir, recargar y pagar	cobros dobles

Campaña: escalonar los logins de las 12:00 y alertar por el p95 del login (el colapso no avisa). Qolqe no se tocó: cada cambio se valida repitiendo correr.sh estres contra 850 / 1,300.

CONCLUSIONES · LAS TRES PREGUNTAS DEL REQUERIMIENTO

Respuestas para la Quincena Qolqe

1 ¿Cumplimos en hora punta?

Sí, con margen: el peor p95 usa el 13 % de su umbral

p95 93 / 128 ms · error 0.00 % · 9.77 req/s · 0 descartadas · 1 h sin deriva

2 ¿Y en el pico de la campaña?

Sí: 304 simultáneos sin degradarse, y se recupera

p95 78 / 123 ms · error 0.00 % · recuperación 80 / 123 ms · 0 descuadres de dinero

3 ¿Hasta cuántos usuarios simultáneos podemos crecer?

850 con 1 CPU (≈ 8.5× la hora punta) · 1,300 con 2 CPU

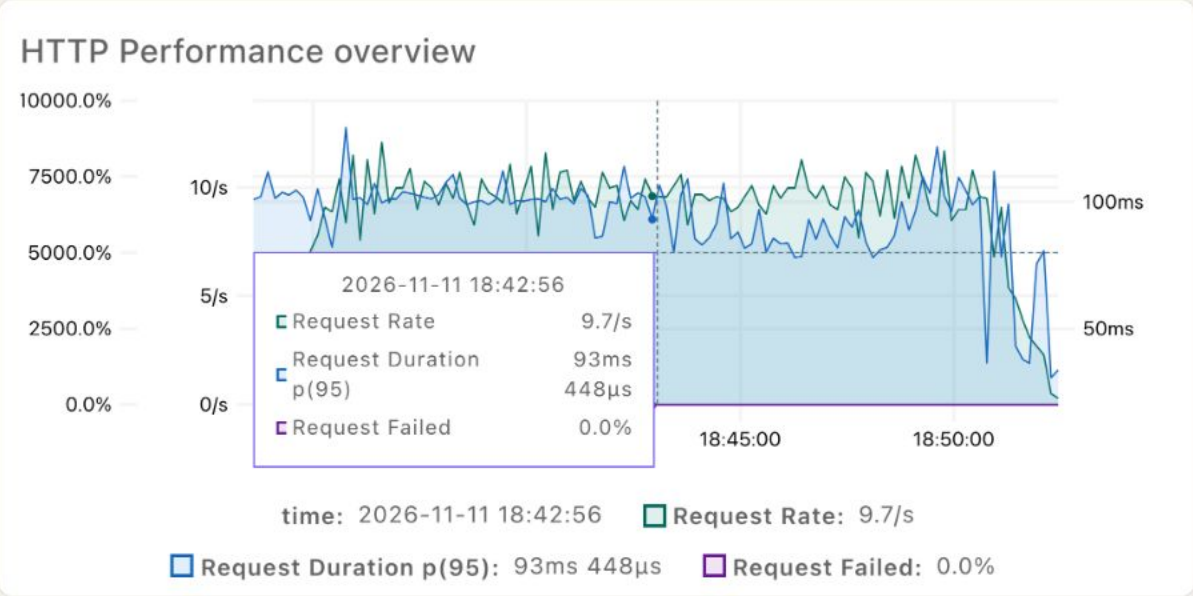
cuello: PBKDF2 del login (CPU) + SQLite con un solo escritor · primera acción: sacar el INSERT de sesión del bloqueo

Riesgo para el negocio: el colapso es brusco, de 78 ms a 13.4 s entre 876 y 946 usuarios, sin aviso previo.

Línea base relativa · evidencia en tests/performance/resultados/ · Gracias. ¿Preguntas?

ANEXO A · LA HERRAMIENTA EN UNA CORRIDA COMPLETA

Reporte de k6 y consola de la carga



reporte.html exportado por k6: a las 18:42:56, 9.7 req/s, p95 de 93 ms y 0.0 % de peticiones fallidas

consola.txt · carga/20261111-1833

```
$ tests/performance/correr.sh carga
Qolqe · escenario carga · 2026-11-11 18:33:26 → 18:52:32
Umbrales (k6 termina con código 99 si alguno falla):
✓ U-01 consultas e inicio de sesión (grupo) p(95)<800      93.2 ms
✓ U-01 login (S-03)                         p(95)<800     104.8 ms
✓ U-01 movimientos (S-03)                   p(95)<800      37.9 ms
  … 11 umbrales más por endpoint, todos ✓ (de 19 en total)
✓ U-02 operaciones de dinero (grupo)        p(95)<1500    128.2 ms
✓ U-02 transferir (S-03)                    p(95)<1500    134.2 ms
✓ U-03 operaciones fallidas (S-06)          rate<=0.01      0.00 %
✓ U-04 throughput ≥ 9.6 req/s               count>=8640      8796
✓ U-05a sesiones descartadas                count==0            0
Veredicto: 19 de 19 umbrales cumplidos con muestras → PASA
errores de rendimiento 0 · errores de datos 0
$ echo $?   # codigo_salida_k6.txt
0
```

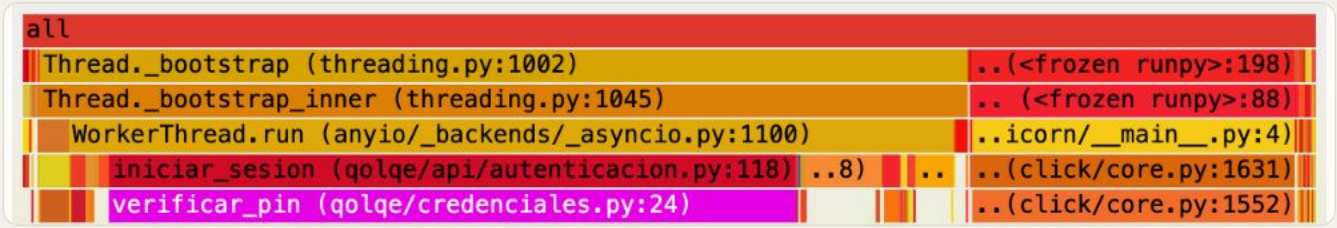
consola.txt: 19 de 19 umbrales → código 0 (11 líneas por endpoint resumidas)

`resultados/carga/20261111-1833`

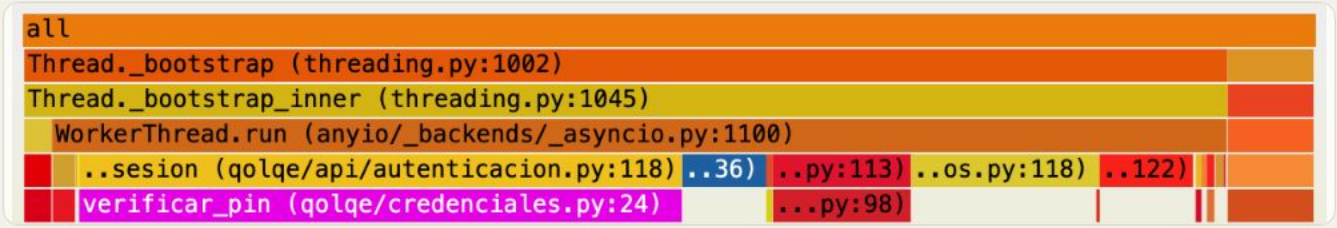
ANEXO B · PERFILES DE PY-SPY Y CPU FRENTE A USUARIOS

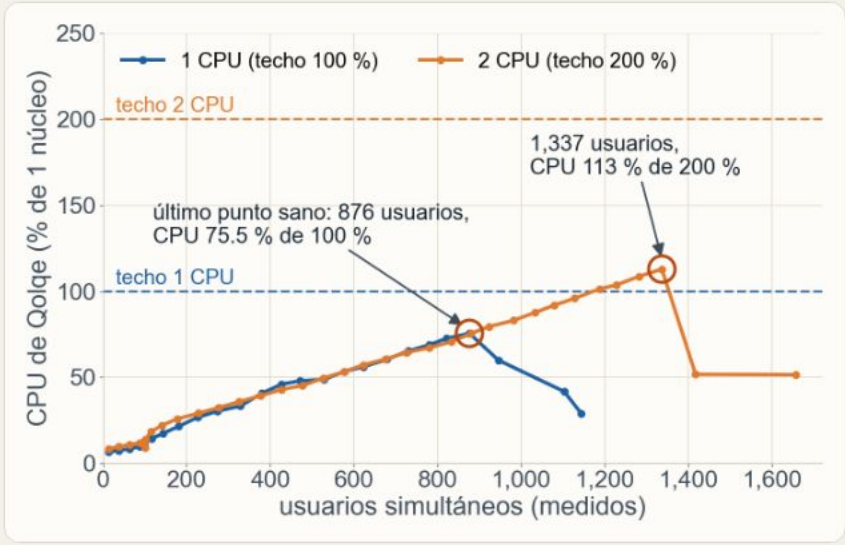
Más evidencia del cuello de botella

Base · 60–120 s (llenado), 1 CPU: verificar_pin 53 %, N+1 de movimientos 5.7 % (línea 118)



Saturación · 2 CPU, 1,000–1,060 s: verificar_pin 46.6 %, movimientos ~32 %, INSERT de sesión 6.5 %





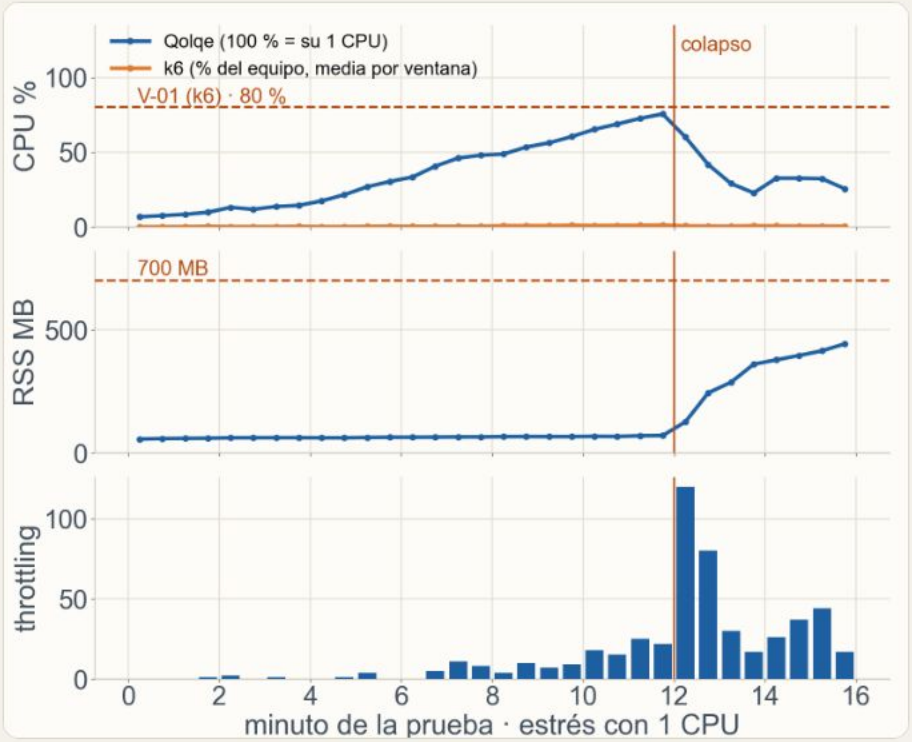
CPU de Qolqe frente a usuarios: con 2 CPU colapsa lejos de su techo

Calibración con 1 VU: 157 ms de CPU por sesión (L_sat ≈ 763); bajo carga, ~103 ms con 1 CPU y ~101 ms con 2 (última ventana que cumple).

`perfil_base.svg · perfil_saturacion.svg · estres/20261111-1940 · escalabilidad/20261111-2103`

ANEXO C · RECURSOS DE CADA CORRIDA

CPU y memoria de Qolqe y de k6 en todas las corridas



Estrés 1 CPU: CPU, RSS y throttling por ventana de 30 s

Recurso	Carga	Pico	Resist.	Estrés 1 CPU	Estrés 2 CPU	Demo
CPU Qolqe, máx. 1 s	23 %	51 %	38 %	101 %	201 %	105 %
RSS de Qolqe, máx.	63 MB	66 MB	63 MB	478 MB	719 MB	110 MB
k6, peor ventana	0.3 %	0.3 %	0.2 %	1.2 %	1.7 %	0.5 %

Lectura: el RSS de Qolqe solo crece cuando las peticiones hacen cola (estrés); en carga, pico y resistencia queda entre 63 y 66 MB. k6 nunca pasó del 1.7 % del CPU del equipo por ventana (V-01 < 80 %).

`ventanas.csv, recursos_k6.csv y veredicto.md de cada corrida`

ANEXO D · POR QUÉ K6

El requerimiento queda escrito en el script

```
// k6/escenarios.js, escenario de carga (extracto; comentarios ampliados)
rate: SESIONES_POR_MIN, timeUnit: '1m',     // M-01: SESIONES_POR_MIN = 50 → 3,000 sesiones/h
duration: dur('17m'),                       // M-12: 2 min de llenado (M-24) + 15 min estables

// umbrales que evalúa k6 en la carga (los arma k6/umbrales.js; carga/…/veredicto.md §6)
'http_req_duration{fase:estable,grupo:consulta}': ['p(95)<800'],   // U-01
'http_req_duration{fase:estable,grupo:dinero}':   ['p(95)<1500'],  // U-02
'operacion_fallida{fase:estable}':                ['rate≤0.01'],  // U-03
'dropped_iterations':                             ['count==0'],    // U-05
```

Si un umbral falla, k6 termina con código 99: la herramienta da el veredicto (paso 1 de S14).

1. **Modelo abierto:** las 50 sesiones/min llegan aunque Qolqe se atrase
2. **Umbral = SLA:** métrica, percentil, límite y carga, por endpoint y grupo
3. **«Sin acumular retrasos» se mide:** dropped_iterations cuenta las sesiones que no empezaron
4. **Liviano:** binario en Go; k6 no pasó del 1.7 % del equipo (V-01)

Limitaciones: sin GUI ni grabador (probamos la API) · la mezcla ponderada se programa (≈ 10 líneas) · no mide CPU ni memoria del servidor: se mide aparte con psutil y docker stats

ANEXO E · MEDICIÓN (ENUNCIADO §3 Y §5)

Medición: qué registra cada corrida

`tests/performance/correr.sh <escenario>`

Equipo libre → Base restaurada → Contenedor nuevo → Monitores → **k6** → Fin y cpu.stat → Verificar dinero → Análisis

k6 2.3.0

Tiempo de respuesta por endpoint, throughput, errores por tipo y sesiones descartadas

`reporte.html`
`muestras.csv.gz`
`resumen.json`

Monitores (1 s)

CPU, RSS, hilos y throttling de Qolqe; docker stats; CPU de k6 (V-01) y del equipo (informativo)

`recursos_qolqe.csv`
`recursos_docker.csv`
`recursos_k6.csv`

py-spy, en estrés

Dónde gasta el CPU Qolqe: perfil en la base y cerca de la saturación; foto de hilos

`perfil_base.svg`
`perfil_saturacion.svg`
`hilos_meseta.txt`

Análisis

Veredicto por umbral (U) y validez (V), tabla de S14 con p50 a p99 y gráficos G1 a G11

`veredicto.md`
`tabla_resumen.md`
`graficos/`

Cada corrida guarda su hora de inicio y fin en corrida.json para cruzar recursos y resultados. Si V-01, V-04 o V-05 no se cumplen, la corrida pasa a descartadas/ con su motivo.